# Operating System Handbook



## 1. Introduction to Operating System

### ★ What is an Operating System ?

An Operating System (OS) is system software that acts as an interface between user and computer hardware.
It manages all resources and provides services for program execution.

**Key Functions of OS**

- ✓ Process Management
- ✓ Memory Management
- ✓ File System Management
- ✓ I/O (Device) Management
- ✓ Security and Protection
- ✓ Resource Allocation
- ✓ User Interface

### ★ Why do we need an OS ?

- It makes the computer system easy to use.
- It manages and allocates resources efficiently.
- It allows multiple programs to run at the same time.
- It provides security and protects data.
- It hides the complexity of hardware from user.

### ★ Types of Operating System

- Batch Operating System
- Time Sharing Operating System
- Distributed Operating System
- Real-Time Operating System

### ★ Components of an OS

→ Kernel
→ Shell / Command Interpreter
→ System Libraries
→ Device Drivers
→ Utility Programs
→ User Programs

### ★ User Mode vs Kernel Mode

| User Mode | Kernel Mode |
|---|---|
| • Limited access to system resources.<br>• Applications run in this mode. | • Full access to system resources.<br>• OS kernel runs in this mode. |

### ★ System Call - How it Works ?

**User Program** — Requests for a service → **System Call** — Request is passed to the OS kernel → **OS Kernel** — Performs the requested task → **System Call Return** — Control and result returned to user program

Note: System call is the interface between a running program and the OS.

# OPERATING SYSTEM HANDBOOK

## 2. PROCESSES AND THREADS

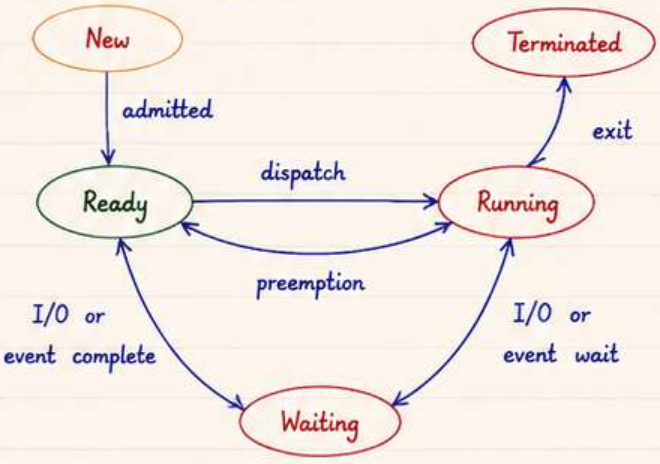
### ★ What is a Process ?

A process is a **program in execution**. It has its own address space, resources, and state of execution.

### ★ What is a Thread ?

A thread is the **smallest unit** of CPU execution within a process. Threads share the code, data and resources of the process.

### ★ Process Life Cycle




### ★ Process Control Block (PCB)

A PCB is a **data structure** maintained by the OS for every process.

| | |
|---|---|
| • Process ID (PID) | • Memory Management Info |
| • Process State | • Accounting Information |
| • Program Counter | • I/O Status Information |
| • CPU Registers | • Priority |
| • CPU Scheduling Info | • etc. |

### ★ Context Switching

Context Switching is the process of **saving** the state of the current process and **loading** the state of another process.

**When does it occur ?**

- When a process goes from running to waiting state.
- When a process is preempted by a higher priority.
- When a time slice expires.
- When a process terminates.

### ★ Process vs Thread

| Feature | Process | Thread |
|---|---|---|
| Definition | A program in execution. | Smallest unit of execution within a process. |
| Memory | Each process has its own separate address space. | Threads share the address space of the process. |
| Resources | Do not share resources. | Share resources of the process. |
| Creation Cost | High (time and memory) | Low |
| Context Switch | Slower | Faster |
| Communication | Through IPC (pipes, sockets, etc.) | Through shared memory |
| Examples | Chrome, VS Code, Notepad | Threads in Chrome, Music Player, etc. |

### ★ Benefits of Threads

- Improves responsiveness of applications.
- Better CPU utilization.
- Easy and efficient communication.
- Reduces memory usage.

### ★ Multithreading Models

- Many-to-One : Many user threads → One kernel thread
- One-to-One : One user thread → One kernel thread
- Many-to-Many : Many user threads → Many kernel threads

# OPERATING SYSTEM HANDBOOK

## 3. CPU SCHEDULING

### ★ What is CPU Scheduling ?

CPU Scheduling is the process of selecting one process from the ready queue and allocating the CPU to it.

The goal is to use CPU efficiently and fairly.

### ★ Types of Schedulers

- Long-Term Scheduler : Selects which processes should be admitted.
- Short-Term Scheduler : Selects next process from ready queue.
- Medium-Term Scheduler : Handles swapping of processes.

### ★ CPU Scheduling Algorithms

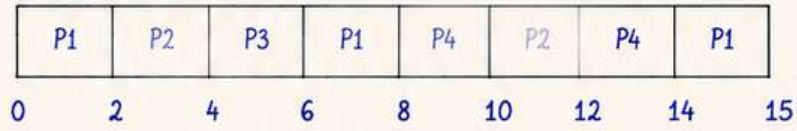
| Algorithm | How it works | Advantage | Disadvantage |
|---|---|---|---|
| FCFS (First Come First Serve) | Processes are executed in the order of their arrival. | • Simple<br>• No starvation | • High waiting time<br>• Convoy effect |
| SJF (Shortest Job First) | Selects the process with the smallest burst time. | • Minimum average waiting time | • Difficult to predict burst time<br>• Starvation for long jobs |
| Priority Scheduling | Selects process with highest priority (lowest number). | • Important jobs get priority | • Starvation for low priority processes |
| Round Robin (RR) | Each process gets a fixed time quantum in cyclic order. | • Good for time sharing<br>• Fair to all | • Context switch overhead<br>• Performance depends on time quantum |
| Multilevel Queue | Processes are divided into multiple queues with fixed priority. | • Efficient<br>• Easy to implement | • Lower queue processes may starve |

### ★ Example : Round Robin Scheduling

Time Quantum = 2

| Process | Arrival Time | Burst Time |
|---|---|---|
| P1 | 0 | 5 |
| P2 | 1 | 3 |
| P3 | 2 | 2 |
| P4 | 3 | 4 |

Gantt Chart :




Note : The smaller the time quantum, the more context switches.

### ★ Scheduling Criteria

- CPU Utilization : Keep CPU busy 100% of the time.
- Throughput : Maximize the number of processes completed per unit time.
- Turnaround Time : Total time taken from arrival to completion.
- Waiting Time : Total time spent in ready queue.
- Response Time : Time from arrival to first response.

**Starvation vs Aging**

- Starvation : A process may never get CPU because other processes are always given priority.
- Aging : Gradually increasing the priority of waiting processes to prevent starvation.

# OPERATING SYSTEM HANDBOOK

## 4. PROCESS SYNCHRONIZATION

### ★ Why Synchronization ?

When multiple processes or threads access **shared data** concurrently, it may lead to **inconsistent** or unexpected results.

Synchronization ensures **correct** and **orderly** access to shared resources.

### ★ Critical Section

The part of a program where shared resources are accessed is called **Critical Section**.

```
do {
   entry section
   critical section
   exit section
   remainder section
} while (true);
```

### ★ Requirements for Critical Section

- ✓ Mutual Exclusion
- ✓ Progress
- ✓ Bounded Waiting

**Note** : Only one process should be in critical section at a time.

### ★ Race Condition

When the result of a program depends on the **timing or sequence** of events caused by concurrent execution of multiple processes.

**Example** : Two processes updating same variable.

### ★ Mutex (Mutual Exclusion)

- A lock that allows only one process to enter critical section.
- If a process wants to enter and mutex is locked, it must wait.

**Mutex Operations**

- lock() → acquire the lock
- unlock() → release the lock

### ★ Semaphore

- An integer variable used to control access to shared resources.
- Two types :
  1. Binary Semaphore (0 or 1)
  2. Counting Semaphore (0 to N)
- Operations : wait() and signal()

### ★ Semaphore Example (Binary)

**Initial value = 1**

| Process A | Process B |
|---|---|
| wait(S); | wait(S); |
| // critical section | // critical section |
| signal(S); | signal(S); |

If S = 0, other process must wait.

### ★ Producer – Consumer Problem


**Solution using Semaphores**

- mutex → ensures only one process accesses buffer at a time.
- empty → counts empty slots in buffer.
- full → counts filled slots in buffer.

### ★ Monitor

- A high-level synchronization construct.
- Encapsulates shared data and the procedures that operate on it.
- Only one process can execute procedures within a monitor at a time.
- Uses condition variables for waiting.

**Monitor Structure**

```
monitor name
{
  shared data;
  condition x, y, ... ;
  procedure P1() { ... }
  procedure P2() { ... }
   ...
}
```

### ★ Readers – Writers Problem

**Problem**

- Multiple readers can read simultaneously.
- Writers need exclusive access.
- Readers and writers cannot access at same time.

**Solution Approach**

- Use of mutex for exclusive access.
- Use of readcount to track number of active readers.
- Ensure no writer is writing when readers are reading.

### ★ Summary

| Mechanism | Purpose | Use When | Key Points |
|---|---|---|---|
| Mutex | Provides mutual exclusion | Only one process in critical section at a time | Simple lock (0 or 1) |
| Semaphore | Controls access to resources | Multiple resources or counting required | Binary or counting |
| Monitor | Encapsulates data + procedures | High-level synchronization needed | Uses condition variables |
| Critical Section | Part of code accessing shared data | Whenever shared data is accessed | Ensure correctness |

# 5. DEADLOCKS

## ★ What is Deadlock ?

A deadlock is a situation where a set of processes are blocked forever, waiting for each other to release resources.

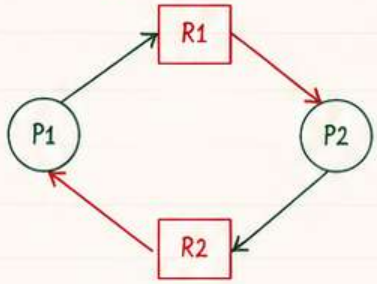
## ★ Four Necessary Conditions

- ✓ Mutual Exclusion – Only one process can use a resource.
- ✓ Hold and Wait – Process holds at least one resource while waiting for others.
- ✓ No Preemption – Resources cannot be forcibly taken.
- ✓ Circular Wait – A set of processes is waiting in a circular chain.

## ★ Resource Allocation Graph

→ Request Edge (Process → Resource)

→ Assignment Edge (Resource → Process)




Cycle exists ⇒ Deadlock occurs.

## ★ Handling Deadlocks

1. Deadlock Prevention : Make sure at least one of the four conditions never holds.
2. Deadlock Avoidance : Allocate resources only if the system stays in a safe state.
3. Deadlock Detection : Allow deadlock to occur, then detect it.
4. Deadlock Recovery : Take actions to recover from deadlock.

## ★ Deadlock Prevention (Ways)

- Make resources sharable where possible.
- Use resource ordering to prevent circular wait.
- Allow preemption of resources.
- Avoid hold and wait by requesting all resources at once.

## ★ Deadlock Avoidance – Banker's Algorithm (Idea)

System State → Check if request keeps system in safe state
→ If yes, allocate
→ If no, wait

It needs info about maximum demand of each process.

## ★ Deadlock Detection (Idea)

- Build a wait-for graph.
- If cycle exists → deadlock.
- Otherwise → safe.

## ★ Deadlock Recovery (Methods)

- Terminate processes (one or more) to break deadlock.
- Preempt resources and allocate to other processes.
- Rollback to a previous safe state.

## ★ Quick Summary

| Concept | Key Idea |
|---|---|
| Deadlock | Processes waiting forever for resources. |
| Necessary Conditions | Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait. |
| Prevention | Ensure at least one condition never holds. |
| Avoidance | Keep system in safe state (Banker's Algorithm). |
| Detection | Allow, then detect using wait-for graph. |
| Recovery | Terminate / Preempt / Rollback to recover. |

# Operating System Handbook

## 6. Memory Management

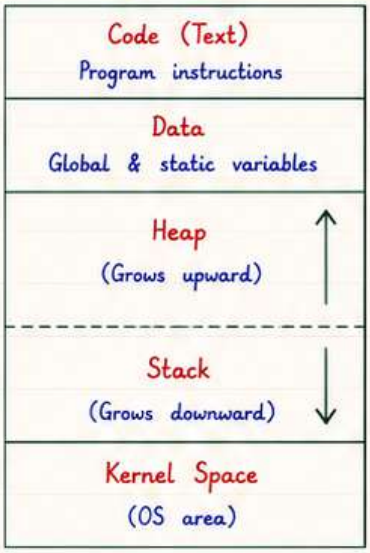
### ★ What is Memory Management ?

Memory Management is the activity of managing the primary memory and coordinating its use among multiple processes.

**Main Objectives**

- ✓ Keep track of memory usage.
- ✓ Allocate and deallocate memory.
- ✓ Ensure protection and isolation.
- ✓ Provide efficient memory utilization.

### ★ Memory Layout (Typical Process)

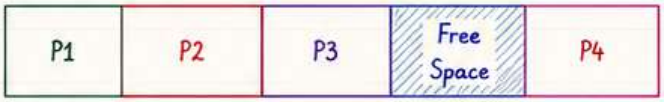



### ★ Logical vs Physical Address

- Logical Address : Generated by CPU (seen by process).
- Physical Address : Actual address in main memory.

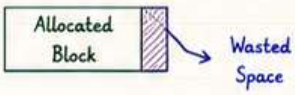

### ★ Contiguous Memory Allocation

Each process is allocated a single contiguous block of memory.




### ★ Fragmentation

- Internal Fragmentation : Wasted space inside an allocated block.

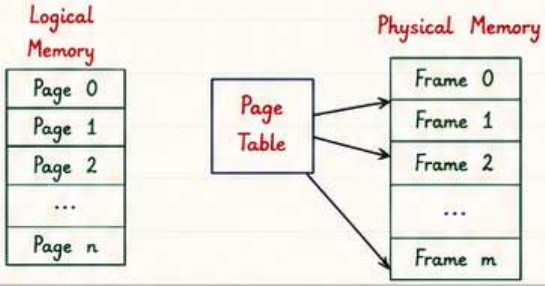



- External Fragmentation : Free memory is broken into small holes.


Many small holes, total free space is enough but cannot allocate big block.

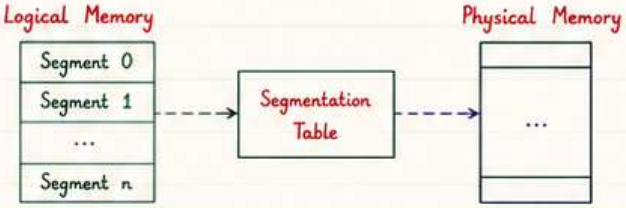
### ★ Paging

- Logical memory is divided into fixed size pages.
- Physical memory is divided into fixed size frames.
- Any page can be placed in any frame.



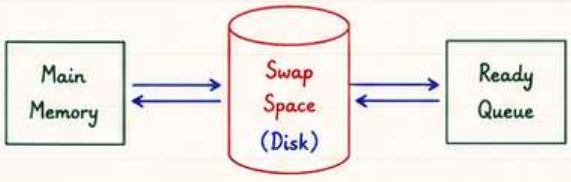

### ★ Segmentation

- Logical memory is divided into variable size segments.
- Each segment represents a logical unit (code, data, stack).
- Requires segmentation table for mapping.




### ★ Swapping

- Entire process is moved between main memory and secondary storage.
- Helps increase degree of multiprogramming.
- But, it is time-consuming.



# OPERATING SYSTEM HANDBOOK

## 7. FILE SYSTEM AND DISK MANAGEMENT

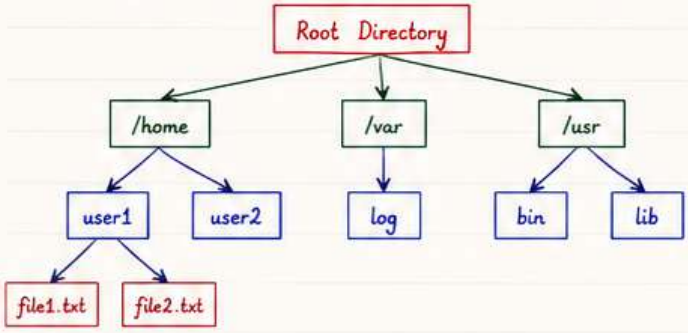
### ★ What is a File System ?

A File System is the way data is organized, stored, named and accessed on secondary storage (like disk).

### ★ File System Functions

- ✓ File creation, deletion and management.
- ✓ Directory management.
- ✓ Provide methods to store and retrieve files.
- ✓ Handle access control and protection.
- ✓ Ensure efficient space utilization.

### ★ File System Structure




### ★ File Types

- Regular File : Stores user data.
- Directory File : Contains a list of files.
- Special File : Represents devices (keyboard, printer, etc.).
- Link File : Points to another file (shortcut).

### ★ File Allocation Methods

| Contiguous Allocation | Linked Allocation | Indexed Allocation |
|---|---|---|
| • Each file occupies contiguous blocks.<br>• Simple and fast access. | • File is stored as linked list of blocks.<br>• No external fragmentation. | • An index block holds pointers to all file blocks.<br>• Supports direct access. |
| F1 \| F1 \| F1 \| F2 \| F2 \| F3 | 5 → 10 → 3 → 8 → EOF | Index Block → 2, 7, 14, 9, 20 |
| (External Fragmentation) | (Pointer overhead) | (Extra space for index block) |

### ★ Directory Structure

- Single Level Directory
- Two Level Directory
- Tree Structured Directory
- Acyclic Graph Directory

**Note** : Most OS uses Tree Structured Directory (like Linux).

### ★ Disk Scheduling Algorithms

Disk Scheduling decides the order in which disk I/O requests are served to minimize seek time.

**Seek Time** : Time taken to move disk head from one track to another.

| Algorithm | How it works | Example (Queue = 98, 183, 37, 122, 14, 124, 65) Head = 53 | Total Head Movement |
|---|---|---|---|
| FCFS | Serves requests in given order. | 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65<br>(98-53)+(183-98)+(183-37)+(122-37)+(122-14)+(124-14)+(124-65) = 640 | 640 |
| SSTF | Serves request with minimum seek time. | 53 → 65 → 37 → 14 → 98 → 122 → 124 → 183<br>= 236 | 236 |
| SCAN (Elevator) | Moves in one direction, then reverse. | 53 → 65 → 98 → 122 → 124 → 183 → 199 → 37 → 14<br>= 331 | 331 |
| C-SCAN | Moves in one direction only, then jumps. | 53 → 65 → 98 → 122 → 124 → 183 → 199 → 0 → 14 → 37<br>= 382 | 382 |
| LOOK | Like SCAN, but does not go to end. | 53 → 65 → 98 → 122 → 124 → 183 → 37 → 14<br>= 296 | 296 |
| C-LOOK | Like C-SCAN, but jumps to last request. | 53 → 65 → 98 → 122 → 124 → 183 → 14 → 37<br>= 322 | 322 |

# Operating System Handbook

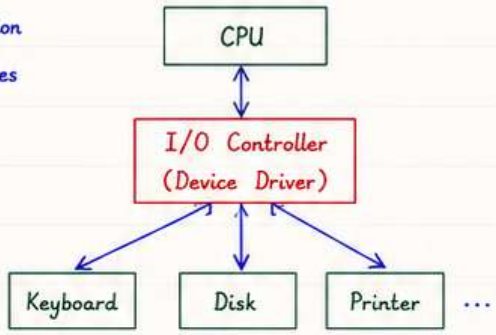


## 8. I/O Management and Operating System Services

### ★ I/O Management

I/O Management handles communication between the system and I/O devices efficiently and safely.

### ★ I/O Structure




### ★ Types of I/O

- Programmed I/O : CPU waits.
- Interrupt Driven I/O : CPU free until interrupt.
- DMA (Direct Memory Access) : Data transfer between device and memory without CPU.

### ★ I/O Techniques

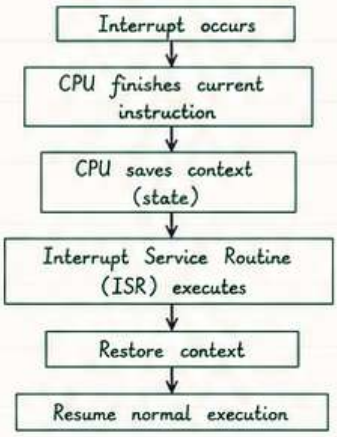
| Technique | Description |
|---|---|
| Buffering | Data is transferred in chunks (buffer) between device and CPU. |
| Caching | Stores frequently used data for faster access. |
| Spooling | Simultaneous Peripheral Operations On-Line. Used for devices like printers. |

### ★ Interrupts

- Interrupt is a signal from hardware to CPU that some event has occurred.
- Types :
  1. Hardware Interrupt
  2. Software Interrupt
  3. Timer Interrupt

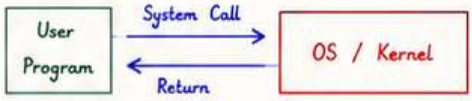
**Flow** : Device sends interrupt → CPU saves state → Interrupt handled → CPU resumes execution.

### ★ Interrupt Handling




### ★ System Calls

System calls provide an interface between user programs and the OS.




**Common System Calls :**

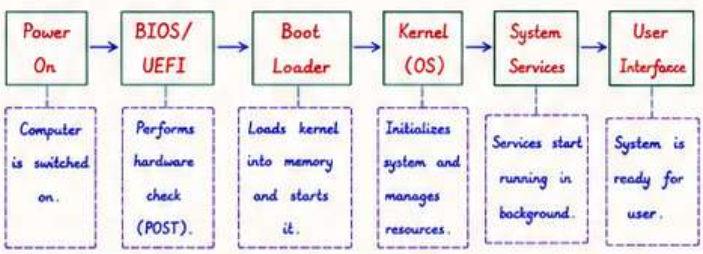
- Process Control : fork(), exec(), exit(), wait()
- File Management : open(), read(), write(), close()
- Device Management : ioctl(), read(), write()
- Information Maintenance : getpid(), time()
- Communication : pipe(), socket(), send(), recv()

### ★ Operating System Services

OS provides services to programs and users.

- User Interface : CLI, GUI, Touch, etc.
- Program Execution : Load program into memory and run it.
- I/O Operations : Uniform access to I/O devices.
- File-System Manipulation : Create, delete, read, write files.
- Communications : Inter-process communication.
- Error Detection : Detect and handle errors.
- Resource Allocation : Allocate CPU, memory, devices.
- Accounting : Keep track of resource usage.

### ★ OS Booting Process




### ★ Quick Summary

| Concept | Key Idea |
|---|---|
| I/O Management | Manages data transfer between OS and devices. |
| Interrupt | Notifies CPU about important events. |
| Buffering | Speeds up I/O by using buffers. |
| Caching | Stores frequently used data in fast memory. |
| Spooling | Allows sharing of I/O devices. |
| System Call | Interface to request OS services. |
| Booting | Process of starting the operating system. |

**Note** : OS is a resource manager and provides a platform for programs to run efficiently.

**Remember** : Good OS = Better utilization, Safety, and Convenience.

# COMMENT PDF TO GET PDF NOTES

## Aman Views

Java • SQL • System Design • DSA
Backend Development • Web Development
QA Testing
Handwritten Notes

TO GET LINK OF ALL OUR NOTES,
CHECK THE LINK IN BIO.